

DevOps and LLMOps Pipeline for a Web-Based Quiz Application

1. Project Overview

The goal of this project was to extend an existing web-based quiz application with a complete DevOps pipeline and an LLM-based feature. The original application was written in HTML, CSS and JavaScript and used The Trivia API for quiz questions.

The extended application supports two modes:

- **Classic Quiz** – questions are retrieved from The Trivia API.
- **AI Generated Quiz** – new questions are generated dynamically using Google Gemini.

The user can choose the category, difficulty and number of questions. The project combines this functionality with continuous integration, continuous deployment, automated testing, infrastructure as code, dependency monitoring and an LLM-specific quality gate.

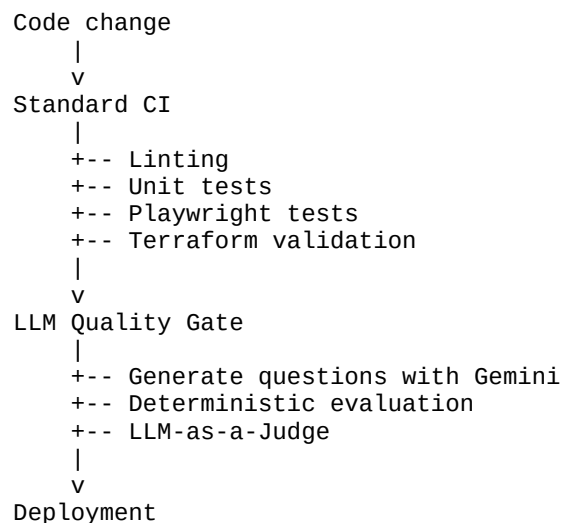
2. Architecture and Development Process

The project is hosted on GitHub and uses GitHub Actions for automation.

The CI pipeline has two main jobs. The first performs standard software checks:

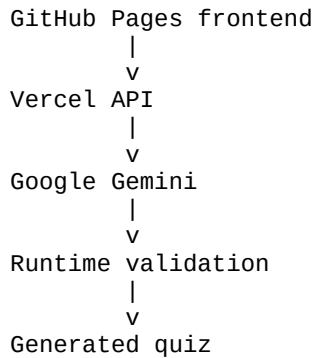
- dependency installation,
- ESLint,
- JavaScript syntax validation,
- unit tests,
- Playwright integration tests,
- Terraform validation.

The second job is the LLM quality gate and runs only after the normal test job succeeds.



The frontend is deployed to GitHub Pages. The CD workflow is triggered only after successful CI, so failing tests or failed LLM evaluation block deployment.

The AI mode requires a backend because the Gemini API key must not be exposed in browser-side JavaScript. A Vercel serverless function is used as the API endpoint:



This keeps the API key private while allowing the static frontend to use Gemini.

3. Testing, Security and Infrastructure

Basic quiz logic was separated into `quizLogic.js` and tested with the built-in Node.js test runner.

Playwright is used for end-to-end testing of both quiz modes. External API calls are mocked during these tests. The Classic Quiz test mocks The Trivia API, while the AI Quiz test mocks the Vercel API response. This makes the browser tests deterministic and avoids failures caused by rate limits or temporary external API problems.

Dependabot is used for dependency and security automation. It monitors npm dependencies and GitHub Actions versions and creates pull requests when updates are available. The Gemini quality job is skipped for Dependabot pull requests because normal repository secrets are not available in those workflows.

Terraform is used as the Infrastructure as Code component. It describes the GitHub Pages configuration and is validated in CI using:

```
terraform init -backend=false
terraform fmt -check
terraform validate
```

This gives the project a declarative and automatically validated infrastructure configuration without introducing unnecessary cloud infrastructure.

4. LLMOps and Quality Gate

Gemini generates quiz questions containing one question, one correct answer, three incorrect answers, a category and a difficulty level.

Structured JSON output is used to enforce the expected response format. However, correct structure does not guarantee correct content, so two additional evaluation layers are used.

Deterministic Evaluation

The deterministic evaluator checks:

- the expected number of questions,
- presence of question text and answers,
- exactly three incorrect answers,
- unique answer options,
- no duplication of the correct answer,
- no obvious answer leakage.

If any check fails, the script exits with a non-zero exit code and the CI job fails.

LLM-as-a-Judge

Gemini is also used to evaluate generated questions. Each question is scored from 1 to 5 for:

- factual correctness,
- clarity,
- distractor quality,
- answerability.

Questions below the required threshold cause the LLM quality job to fail.

Generated questions are stored as GitHub Actions artifacts so failed outputs can be inspected later.

The backend also performs runtime validation before AI-generated questions are returned to the user. This means quality control is applied both before deployment and while the application is running.

5. Design Decisions, Limitations and Conclusion

The frontend was kept as plain HTML, CSS and JavaScript instead of introducing a build framework such as Vite. The existing application did not require a build step, so adding one would increase complexity without a clear benefit.

Docker was also not used because the frontend is static and the AI backend is serverless. Neither part requires a persistent containerized server.

A key design choice was to separate deterministic browser tests from real LLM calls. Playwright uses mocked responses, while the dedicated LLM quality job communicates with Gemini. This keeps normal tests fast and reliable while still evaluating the real model in CI.

The main limitation is that LLM output is non-deterministic. Even with structured output, deterministic validation and an LLM judge, factual correctness cannot be guaranteed completely. The AI mode also depends on the availability and rate limits of Gemini. In addition, Gemini is used both to generate and evaluate questions, which may introduce shared model bias; an independent evaluator could provide stronger validation in a production system.

Overall, the project extends a simple quiz application into a complete DevOps and LLMOps workflow. GitHub Actions provides CI/CD automation, Playwright and unit tests provide software quality checks, Dependabot provides dependency automation, Terraform provides infrastructure validation, and Vercel provides a secure backend for Gemini. The project demonstrates how traditional DevOps practices can be extended to applications that include non-deterministic LLM components while still maintaining automated quality gates and controlled deployment.